

Mamba vs Convolutions

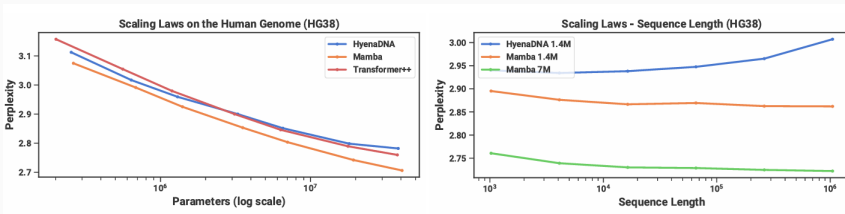
Adityavardhan Srivastava

June 21, 2026

Indian Institute of Science, Bangalore

Initial Verification of Mamba

Objective: Before addressing our primary problem statement, we first tried to verify the paper by replicating Mamba's baseline on the HG38 genome. The models trained for this section are all single Mamba blocks. The graph given in the paper:



We set the sequence length to $2^{10} = 1024$ similar to the paper and trained the following models:

Micro-Scale Architecture	Number of Parameters	Validation PP ($\text{seq_len} = 1024$)
Mamba ($d_{\text{model}} = 64$)	33,285	2.8645
Mamba ($d_{\text{model}} = 128$)	117,765	2.7971

Despite operating at a fraction of the literature's parameter budget ($\sim 10^5$ vs 10^7), we successfully hit the expected absolute Perplexity (PP) of ~ 2.8 .

Initial Verification of Mamba

While absolute performance matched, our results actively diverged from published scaling laws. While the paper describes that the perplexity decreases slightly with increasing `seq_len`, our results show a non-uniform pattern.

Sequence Length ($d_{model} = 128$)	Validation PP	Observation
256	2.8370	Baseline
1024	2.7904	Decreases
2048	2.8861	Increases drastically
4096	2.8157	Decreases again

The divergence from the laws given in the paper can be due to the following reasons:

- **Architectural reduction:** The size of the model was severely reduced to fit the memory constraints.
- **Partial Dataset used:** Due to memory and time limitations, the training and validation data was taken for just a single chromosome.

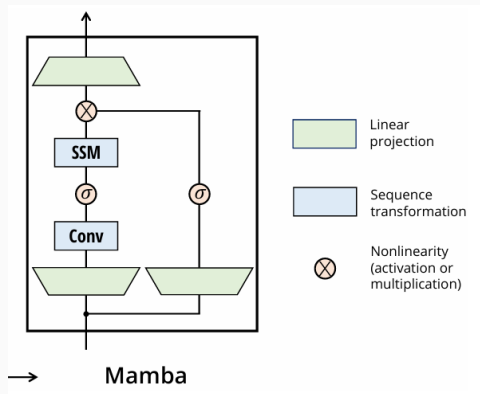
Even though the models failed to capture the scaling laws accurately, the values of perplexity obtained were as predicted by the paper which gives validation to the models being used.

Architecture: Pure Mamba (State Space Model)

Mamba transitions away from attention mechanisms by using an LSSL along with a hardware aware Selective Scan Algorithm, allowing for linear time complexity $O(N)$.

PyTorch Implementation:

- Sequences are flattened and the tokens are normalized before passing through any of the Mamba blocks.
- The sequence is passed through the Mamba block displayed to the right. (mamba_ssm)
- The weights of the linear layers, the convolution layer and the SSM layer (A, B, C and Δ) are learned through backpropagation. (mamba_ssm)
- The output of the block is combined with a skip connection through element wise addition.
- Such blocks are generally stacked on each other for more complex models.



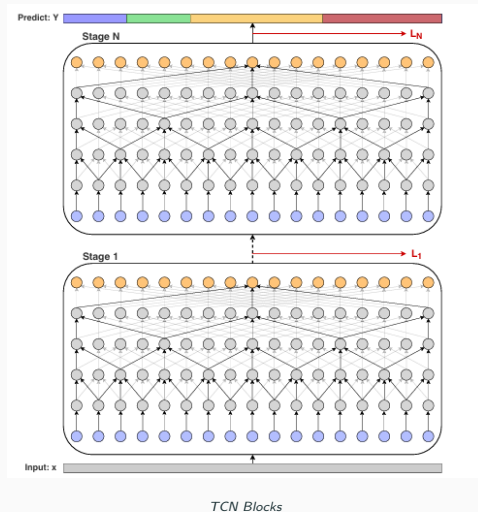
Mamba Block Structure

Architecture: Pure TCN (Temporal Convolution)

Temporal Convolutional Networks (TCN) are a special type of CNNs that use dilations and stacked layers to collect the information distributed along a sequence.

PyTorch Implementation:

- Sequences are flattened and the tokens are normalized before passing through any of the TCN blocks.
- The sequence is passed through one such TCN block displayed to the right.
- The weights of the linear layers and the convolution layer are learned through backpropagation.
- The number of layers per block are chosen such that each token can access the entire sequence.
- Such blocks are generally stacked on each other for more complex models.

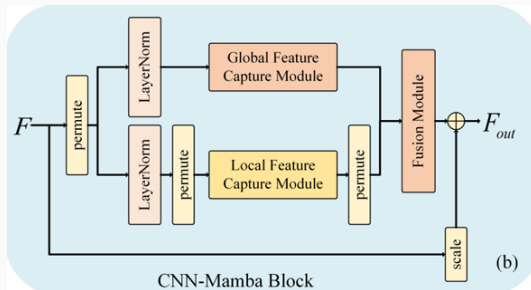


Architecture: Parallel Hybrid (Mamba + TCN)

Combine the global, long-range context of State Space Models with the local, high-frequency feature extraction of CNNs.

PyTorch Implementation:

- Input is flattened to 1D, normalized and processed via the Mamba block and a TCN layer in parallel.
- The outputs are fused with the skip connection (scale = 1) by component wise addition. This gives the final output of the unit of the hybrid parallel model.



Experiment 1: sCIFAR & Parameter Constraints

The sCIFAR Dataset: Sequential CIFAR-10 flattens 32×32 images of the original CIFAR-10 dataset into a 1024-step sequence. This forces the model to treat visual data as a long time-series, testing its ability to retain long-range dependencies.

Methodology: Strict Parameter Budgeting

To ensure a fair comparison, the total parameter count across architectures was strictly capped at $\sim 468,000$. This was done through manipulating the hidden dimension width (d_{model}).

Architecture	TCN layers/ Mamba Blocks	d_{model}	Total Parameters
Pure Mamba	4	128	468,746
Pure TCN	8	139	468,996
Hybrid Parallel	4	106	465,774

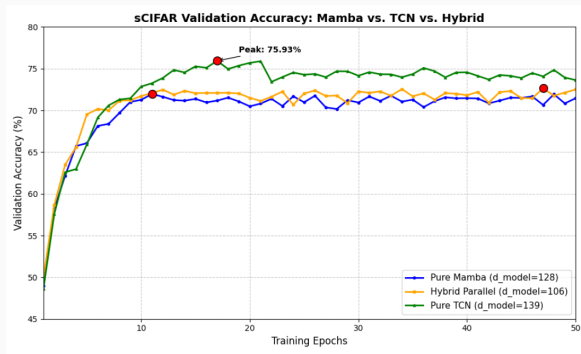
Experiment 1: Results & Analysis

Peak Validation Accuracy:

- Pure TCN: 75.93%
- Hybrid: 72.63%
- Pure Mamba: 71.96%

Observations:

1. The TCN model performs better than both the other models with a clear margin. As this model is non-causal, it can access both the past and the future of the temporal sequence. This allows the TCN model to look at the dataset item within its spatial context.
2. The pure Mamba and the hybrid model performed similarly with the hybrid winning marginally. The advantage of gaining spatial context by the hybrid is balanced by the decreased dimension of pixel encodings.



Experiment 2: SpeechCommands & Temporal Capacity

The SpeechCommands Dataset: This task evaluates the architectures on 1D audio waveform classification across 35 distinct classes from the `torchaudio.dataset` library. The audio data is processed into a strict 4000-step temporal sequence, testing the models' capacity to handle extreme long-range, causal dependencies.

Methodology: Parameter Budget & Width Restriction

To maintain a strict parameter budget for fair comparison similar to experiment 1, the parameters were kept equal for all 3 models. This was again done through the manipulation of the hidden dimension.

Architecture	Layers / Blocks	<i>d_{model}</i>	Total Parameters
Pure Mamba	8	64	264,547
Pure TCN	11	89	267,658
Hybrid Parallel	8	53	258,940

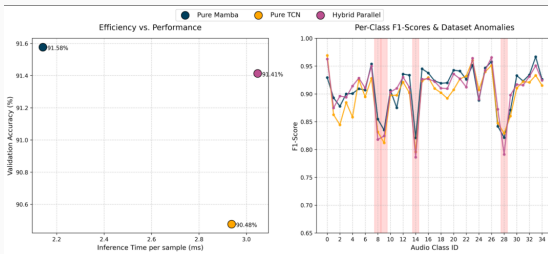
Experiment 2: Results & Analysis

Peak Accuracy & Speed:

- **Pure Mamba:** 91.58% (2.05 ms/sample)
- **Hybrid:** 91.41% (2.97 ms/sample)
- **Pure TCN:** 90.48% (2.92 ms/sample)

Observations:

1. The pure Mamba model acquires both speed and accuracy and is able to be the most optimal model in this case. The 4000 step long audio sequence is a strictly causal 1D sequence, which allows the architecture of Mamba to shine.
2. All three architectures exhibit similar F1-score crashes on specific audio classes (e.g., 8, 9, 14, 28). As the models (TCN and Mamba) are based on very different mathematical approaches, both of them failing to learn certain classes can point to ambiguities (such as noise or lower number of data items) in those classes.



U-Net Architecture for Image Defocus Deblurring

Foundational Architecture:

We took inspiration from the architecture proposed in the Dual-Branch CNN-Mamba Method for Image Defocus Deblurring.

- Image deblurring is a complex 2D spatial deconvolution process that requires both local high frequency edge reconstruction and global low frequency structure retention.
- The authors propose a dual-pathway block, utilising a bidirectional CNN branch for local features, and a State Space Model (Mamba) branch for global context.

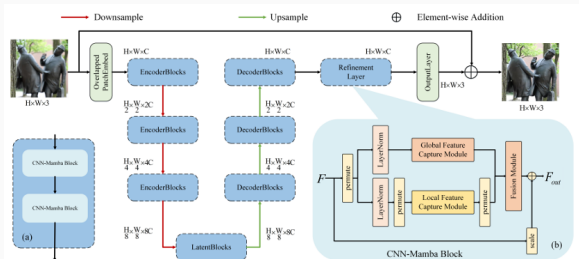


Figure 3. Architecture of CMDDNet for image defocus deblurring. CMDDNet adopts the classic U-Net framework. The encoder, latent layer, and decoder are constructed by stacking two CNN-Mamba blocks as illustrated in (a). The structure of each CNN-Mamba block is depicted in (b).

MODular architecture

To test the performance and necessity of the dual-branch approach, we implemented a custom U-Net capable of dynamically swapping its core computational blocks:

- **Pure CNN Block:** Standard bidirectional convolutions.
- **Pure Mamba Block:** Flattened 1D sequence processing.
- **CNN Mamba (Hybrid) Block:** The dual pathway fusion.

Note that we use just a single block per `EncoderBlocks` and `DecoderBlocks`, as opposed to 2 blocks in serial as shown in the paper.

Critical Modification: Micro-Batch Normalization

Due to VRAM constraints, training was limited to `batch_size = 2` for safety. To prevent the introduction of noise into the model, we replaced the CNN branch's standard `BatchNorm` with `GroupNorm` (`groups = 8`). This stabilizes the micro-batch training while allowing distinct feature maps (e.g., edge detectors, color gradients, etc.) to normalize independently. We also avoid `LayerNorm` as it normalizes across the entire collection of channels and pixels, intermixing the values of separate feature maps and locations.

Experiment 3: The DPDD dataset, validation metrics and parameter budgeting

The DPDD Dataset: The Dual-Pixel Defocus Deblurring dataset provides heavily blurred source images mapped to perfectly sharp targets, to test models on the defocus deblurring task which includes spatial restoration of high frequency features.

Validation Metrics:

- **PSNR:** Peak Signal to Noise Ratio. This measures pixel wise mathematical accuracy. Higher is better.
- **LPIPS:** Learned Perceptual Image Patch Similarity. It is a critical metric for deblurring, as it evaluates structural integrity and texture the way human vision does using the results of another pretrained model. Lower is better.

Methodology: Strict Parameter Budgeting

To ensure a fair comparison, parameters were locked at $\sim 4.3\text{M}$. Because the Hybrid block runs two parallel pathways, it suffers a penalty to maintain fair number of parameters. This is done by manipulating the value of `base_c`, which is the number of channels we generate from the first convolutional layer.

Architecture	<i>base_c</i>	Parameters
Pure CNN	88	4,314,467
Pure Mamba	64	4,286,659
Hybrid Parallel	56	4,325,723

Experiment 3: Results & Analysis

Metrics and Comparison:

- Mamba wins on peak PSNR (22.29). Hybrid (22.21) and CNN (21.86).
- Hybrid wins stable LPIPS floor (~ 0.438) and peak LPIPS.

Observations:

1. Despite operating at a reduced capacity ($base_c = 56$), the Hybrid architecture produces the most perceptually realistic images (lowest LPIPS).
2. **Late-Stage Degradation:** While Pure Mamba spikes early (large learning rate early), its LPIPS degrades and oscillates around ~ 0.452 in the final 10 epochs. The Hybrid model establishes a lower representational floor, supporting the hypothesis that global state space models require local CNN support for optimal spatial physics tasks.

